Peterson's Algorithm

A Software-Based Synchronization Solution for Multi-threaded Java Environments

Deshan Siriwardhana | Indumini Wijesinghe | Thilakshi Nethmini

The Critical Section Problem

A fundamental challenge in concurrent programming where threads access shared resources simultaneously.

CORE REQUIREMENTS FOR A SOLUTION

Mutual Exclusion

Only one thread can execute its critical section at a time, ensuring data integrity.

Progress

If no thread is in the CS, a thread wishing to enter can proceed without unnecessary delay.

Bounded Waiting

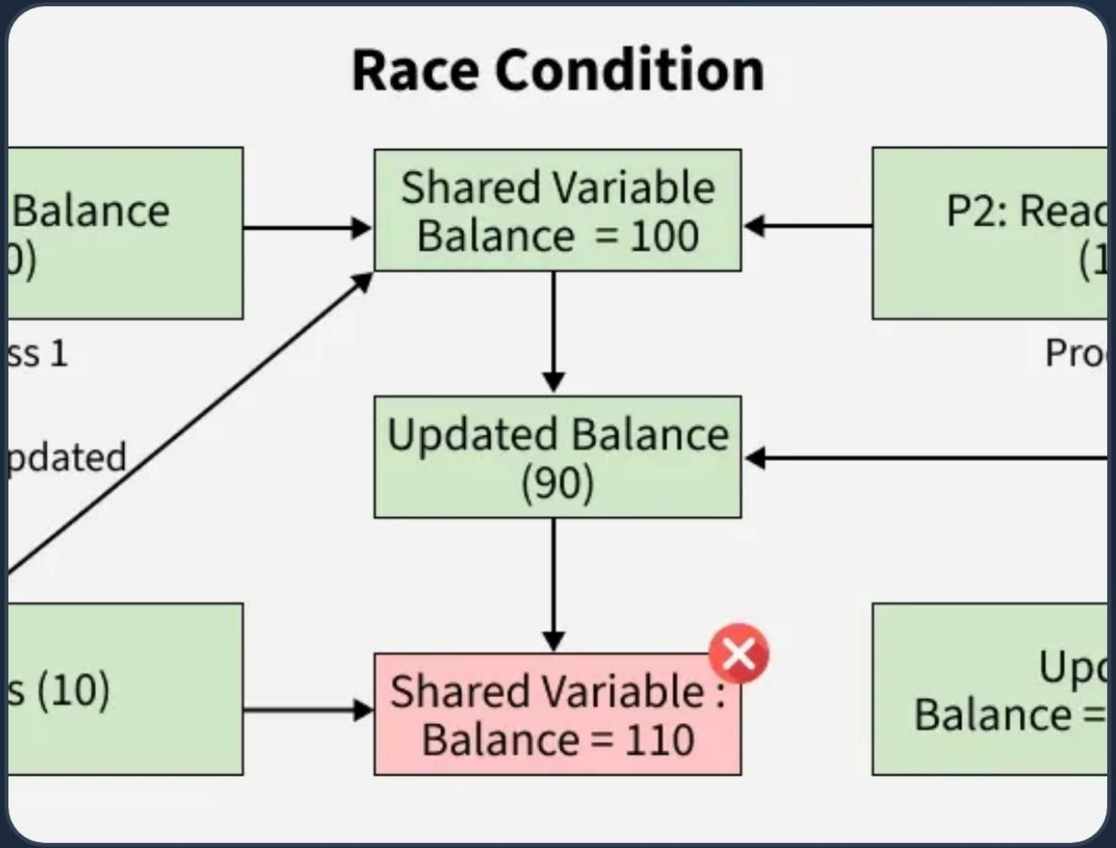
No thread should wait indefinitely; there's a limit on how many times others can enter first.

THE PROBLEM: RACE CONDITION

Unsynchronized Execution

When multiple threads modify shared data simultaneously, **Race Conditions** occur. This leads to data corruption and inconsistent states.

In our demo, the operation `count++` is not atomic. It involves a **Read-Modify-Write** cycle at the CPU level.



LOST UPDATES DEMONSTRATED

187,741

ACTUAL FINAL COUNT

200,000

EXPECTED COUNT

Status: Failed! Data inconsistency detected due to lack of synchronization.

Implementing Peterson's Solution

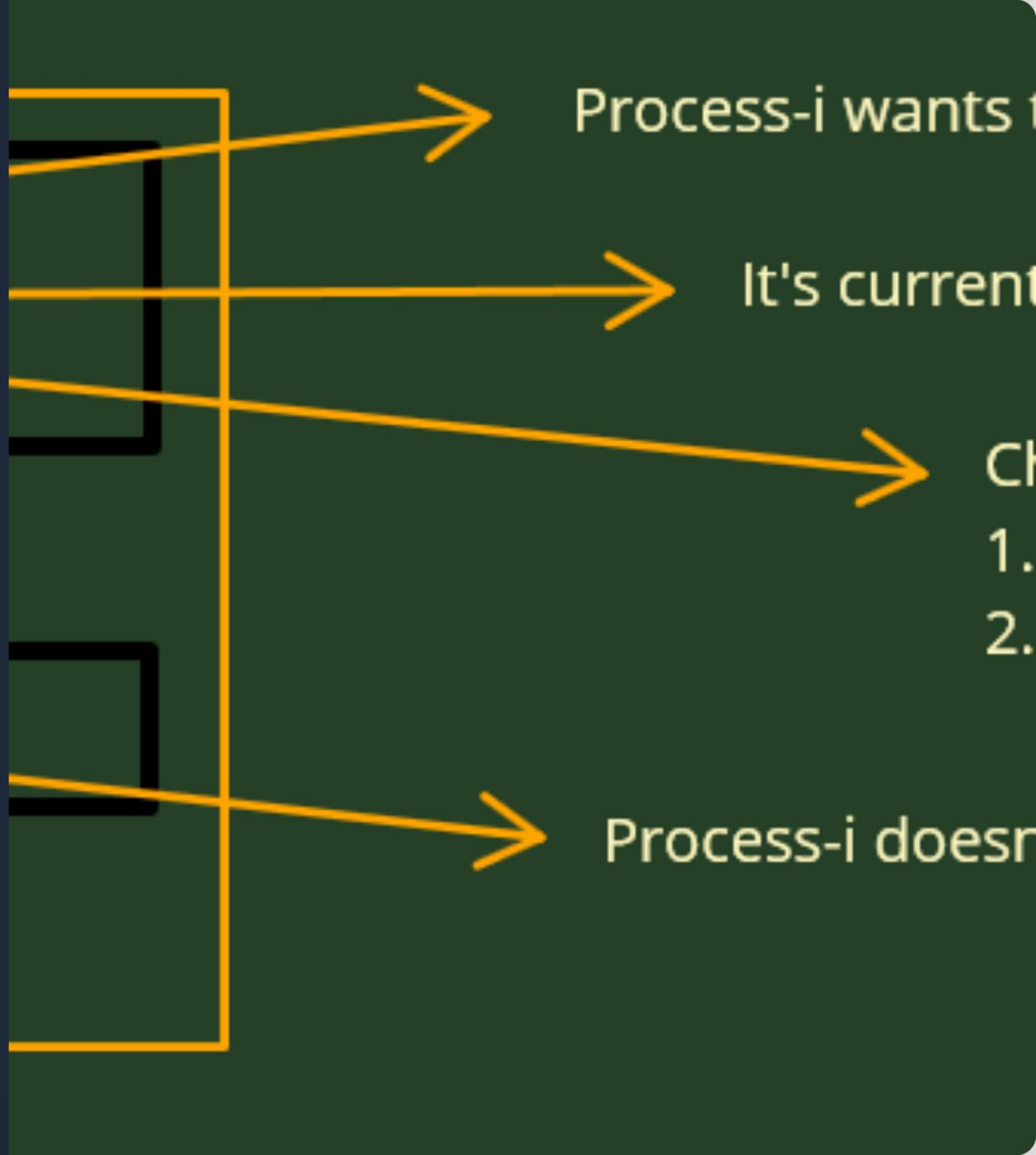
A robust software-only approach for two-process synchronization.

THE ALGORITHM LOGIC

Peterson’s algorithm uses two shared variables to coordinate entry into the critical section:

- **Flag[2]:** Indicates a thread's desire to enter.
- **Turn:** Acts as a tie-breaker, determining whose turn it is.

By yielding the turn immediately after expressing interest, it prevents deadlocks and ensures fairness.



JAVA IMPLEMENTATION DETAILS

- **Volatile Keyword:** Essential for memory visibility. It ensures that `flag` and `turn` updates are instantly visible to both threads.
- **Enter Section:** A thread sets its flag to true, hands the turn to the opponent, and waits as long as the opponent is interested and it's their turn.
- **Exit Section:** Once finished, the thread resets its flag to false, immediately unblocking the other waiting thread.

CODE WALKTHROUGH & LOGIC

Component	Implementation Logic
Shared State	`volatile boolean[] flag = new boolean[2]; volatile int turn = 0;`
Entry Section	`flag[i] = true; turn = j;` (Setting intent and yielding turn)
Wait Condition	`while (flag[j] && turn == j);` (Busy-wait until safe)
Critical Section	`count++;` (The shared resource update)
Exit Section	`flag[i] = false;` (Allowing other processes to proceed)

PERFORMANCE & SUCCESS COMPARISON

Result: Peterson's Algorithm achieves 100% data consistency.

Conclusion & Reflections

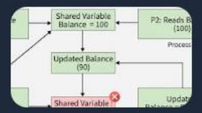
Peterson's Algorithm remains an elegant, hardware-independent demonstration of software-based mutual exclusion.

Questions?

Thank you for your attention.

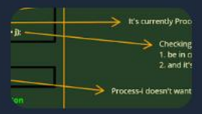
Project by: Deshan Siriwardhana | Indumini Wijesinghe | Thilakshi Nethmini
CINEC Campus | Operating Systems (IT 2106)

IMAGE SOURCES



https://media.geeksforgeeks.org/wp-content/uploads/20250829161248232387/racecondition.webp

Source: www.geeksforgeeks.org



https://miro.medium.com/1*Uii-lQfIwQ6LBN8-Uo9yEw.png

Source: medium.com